Option initiation à la recherche, partie VESONTIO
Master 1 informatique ISL

Alain Giorgetti

Université Marie et Louis Pasteur, institut FEMTO-ST, CNRS, F-25000 Besançon, France

Version du 11 février 2026

Introduction

Objectifs

1. Vivre une expérience de recherche scientifique en équipe, sur un problème (a priori) ouvert ("science en direct")
2. Manipuler divers outils du chercheur
   - Travail collaboratif
   - Veille bibliographique
   - Rédaction et communication scientifiques
   - . . .
3. Éventuellement, faire émerger et initier un sujet de projet + stage de recherche en master 2

Équipe VESONTIO

- Formalisation de concepts, de problèmes ou d'algorithmes
- Formalisation de leurs propriétés
- Vérification par test automatique et/ou preuve formelle
- Outils spécifiques : Prolog, solveurs SMT, prouveurs

Sujets d'étude

1. IAP : IA et Preuve formelle
2. ARIA : Accompagnement de la Recherche par Intelligence Artificielle, projet M2 professionnel
3. RAISER (Reasoned use of Artificial Intelligence by Software Engineers and Researchers), Maryam Assmar
4. Qatlas : Découverte de configurations quantiques contextuelles optimales, Wilfried Kamga
5. SATsym : problèmes SAT symétriques
6. Bij : preuves formelles de bijections réciproques
7. SAW : Chemins auto-évitants

À faire avant le 2e CM :

1. Former un Groupe de Travail (GT) sur un de ces sujets
2. Dans ce sujet, identifier (au moins) une Question de Recherche (QR) ouverte

Questions méthodologiques

(GL appliqué à la recherche en informatique)

- Comment représenter formellement un problème donné ?
- Comment générer/énumérer/compter des solutions ?
- Quelles propriétés doivent/peuvent être vérifiées ?
- Comment automatiser (davantage) la vérification de ces propriétés ?

⇝ Produit d'activité 1 : une discussion par GT dans le forum Moodle

- Sujet choisi
- Membres du GT
- Rôle de chaque membre
- Question de recherche étudiée

Planning

12/2/26	Cours 1	Présentation d'exemples de questions de recherche ouvertes (💡 . . .)
Constituer un GT (4 ou 5 membres) sur au moins une QR liée à un sujet		
2/3/26	Cours 2	Présentation d'outils du chercheur
Approfondir ces outils en autonomie		
13/3/26	TP 1	Pratique des outils (Zotero, édition collaborative) sur le *sujet choisi* : explicitation et reformulation du sujet, découpage en tâches, rédaction de la problématique
Collecter et étudier les références, commencer les recherches		
18/3/26	TP 2	Partage des résultats, rédaction scientifique collaborative
Rédiger une note de lecture des références et une explication du code produit		
19/3/26	TD 1	Finalisation de la rédaction collaborative : rapport écrit + slides de présentation orale (*beamer*)
31/3/26	*Tutorial*	*in English, for SSE and voluntary students*
Finaliser rapport écrit et slides de présentation, déposer sources et code produit		
29/4/26	TD 2	Exposé court des travaux et résultats de chaque équipe

Travaux attendus et critères d'évaluation

1. État de l'art ciblé (20%)
   - 1.1 recherche de références, avec Zotero
   - 1.2 étude de références, pour répondre à des questions précises
   - 1.3 rédaction d'une synthèse des informations trouvées (LaTeX + bibtex + PDF)
2. Mise en œuvre d'outils du chercheur (40%)
   - 2.1 installation
   - 2.2 expérimentation avec des exemples existants
   - 2.3 adaptation au sujet choisi
   - 2.4 explication écrite du code original produit (LaTeX + PDF)
3. Résolution de la problématique de recherche choisie (40%)
   - 3.1 niveau d'avancement
   - 3.2 gestion des difficultés
   - 3.3 pertinence du partage et de la gestion des tâches
   - 3.4 efficacité collaborative du groupe
   - 3.5 qualité de la communication

⇝ Organiser le travail, échanger dans au moins une discussion par groupe, dans le forum dédié sur le site Moodle `https://moodle.univ-fcomte.fr/course/view.php?id=24831` (en particulier, **avant le CM 2 du 2/3/26**, déclarer sur ce forum le sujet choisi et la composition de votre groupe)

Plan

1. Introduction
2. IA et preuve formelle
3. Accompagnement de la Recherche par Intelligence Artificielle
4. Reasoned use of Artificial Intelligence by Software Engineers and Researchers
5. Découverte de configurations quantiques contextuelles optimales
6. Problèmes SAT symétriques
7. Bijections réciproques
8. Chemins auto-évitants
9. Outils du chercheur

Plan

1. Introduction
2. IA et preuve formelle
3. Accompagnement de la Recherche par Intelligence Artificielle
4. Reasoned use of Artificial Intelligence by Software Engineers and Researchers
5. Découverte de configurations quantiques contextuelles optimales
6. Problèmes SAT symétriques
7. Bijections réciproques
8. Chemins auto-évitants
9. Outils du chercheur

IA et preuve formelle - Introduction

(crédits : présentation d'Abdelhak-D. Seriai, réunion défis du GDR GPL, 3 mars 2025)
IA et GL, trois périodes

- IA symbolique (1950-70)
  Systèmes experts pour analyser le code, détecter des anomalies, résoudre des problèmes complexes de planification grâce à des algorithmes basés sur des règles, etc
- IA connexionniste, puis probabiliste (1980-2010)
  Prédire les coûts, détecter les anomalies, maintenance prédictive, systèmes de recommandation pour les développeurs, etc
- Deep Learning et LLM (2010-)
  Prédiction de défauts logiciels, génération de code, analyse automatique de documents, etc
  Grands modèles de langage (LLM) : génération automatique de code, revue de code assistée, documentation automatique, génération de tests logiciels, etc

IA et preuve formelle - IA4GL

IA4GL : Approches intégrées basées sur l'IA pour la résolution des problèmes du génie logiciel
Comment utiliser l'IA générative pour ie GL : Utilisation des grands modèles de langage (LLMs) ?

- Ingénierie des prompts (requêtes à ces LLMs) et RAG (*Retrieval-Augmented Generation*) sont, par exemple, des pratiques qui peuvent permettre d'améliorer différentes phases du processus de développement logiciel : génération de code mais aussi ingénierie des besoins
- Programmation en binôme assistée par l'IA (IA générative/LLM) : *IA-Supported Pair programming*
- Réparation automatisée du code généré par l'IA (*Automated Repair for IA-generated code*)

💡 Quel est l'impact de l'IA générative sur les méthodes de preuve formelle ?

Plan

1 Introduction
2 IA et preuve formelle
3 Accompagnement de la Recherche par Intelligence Artificielle
4 Reasoned use of Artificial Intelligence by Software Engineers and Researchers
5 Découverte de configurations quantiques contextuelles optimales
6 Problèmes SAT symétriques
7 Bijections réciproques
8 Chemins auto-évitants
9 Outils du chercheur

Contexte

ARIA : Accompagnement de la Recherche par Intelligence Artificielle

- Problème : Logiciels de recherche souvent abandonnés après le stade *Proof of Concept* (PoC)
- Causes
  - Manque de temps et de ressources pour la maintenance
  - Activités d'ingénierie peu valorisables en recherche
- Solution envisagée : soutien par agents IA
  - Automatiser le développement et la maintenance
  - Prolonger la durée de vie des logiciels
  - Améliorer leur qualité

Objectifs et méthodologie

Objectifs principaux

- Développer un cadre d'assistance par IA pour logiciels de recherche
- Intégrer une réflexion éthique sur l'utilisation des IA (confidentialité, impact environnemental, ...)

Méthodologie

1. Analyse de l'état et des besoins de PoC existants (Audacy, SANTE, AutoCheck ou Qontextium)
2. Identification des tâches automatisables
3. Prototypage d'agents IA responsables
4. Évaluation de l'efficacité et de l'impact écologique
5. Documentation complète

Projet M2 professionnel 2025-26, choisi par Amayas Seffar et Sami Guetarni, soutenance le 24/2/26 de 10h à 11h en 404C
Exemple de question ouverte : Intégration systématique (outillée ?) de considérations éthiques

Plan

1 Introduction
2 IA et preuve formelle
3 Accompagnement de la Recherche par Intelligence Artificielle
4 Reasoned use of Artificial Intelligence by Software Engineers and Researchers
5 Découverte de configurations quantiques contextuelles optimales
6 Problèmes SAT symétriques
7 Bijections réciproques
8 Chemins auto-évitants
9 Outils du chercheur

RAISER

Reasoned use of Artificial Intelligence by Software Engineers and Researchers

Objectifs principaux

- Étudier l'impact de l'IA générative sur les méthodes formelles de vérification logicielle
- Explorer comment l'IA peut compléter les approches rigoureuses (preuves formelles, model-checking)

Enjeux éthiques, approche raisonnée

- Développer des solutions souveraines et ouvertes (approches locales, open-source, frugales)
- Impact écologique de l'IA (consommation énergétique, eau, ressources numériques)

Méthodologie

Étapes clés

1. Collecte et analyse d'articles récents d'application de l'intersection IA générative à la vérification formelle
2. Étude des outils existants (ex. : NeuroSAT [SB18] pour la résolution de problèmes logiques)
3. Développement de prototypes combinant IA et méthodes formelles
4. Évaluation des approches sous contraintes écologiques

Références

- Conférences : S2RAI, JRAF [JRA25]
- Articles : [BCG24], ...

[BCG24] Lasse Blaauwbroek, David Cerna, Thibault Gauthier, Jan Jakubův, Cezary Kaliszyk, Martin Suda, and Josef Urban.
Learning guided automated reasoning : A brief survey.
arXiv preprint 2403.04017, 2024.
https://arxiv.org/abs/2403.04017.

[JRA25] JRAF : Journées sur la Recherche en Apprentissage Frugal, last visit in August 2025.
https://edge-intelligence.imag.fr/jraf.html.

[SB18] Daniel Selsam and Nikolaj Bjørner.
Neurosat : Learning a sat solver from single-bit supervision.
arXiv preprint arXiv :1802.03685, 2018.

Contributions

Partie 2.1 de [AGOT26]

Contribuer à une version étendue (*position paper*) avant le 31/5, pour LNCS ?

Questions de recherche ?

- ...
- ...
- ...

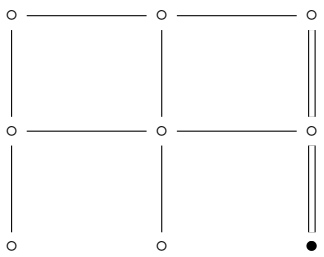
Contact : `maryam.assmar@edu.umlp.fr`

[AGOT26] Maryam Assmar, Alain Giorgetti, Meriem Ouederni, Sylvie Trouilhet.
Challenges for Engagement between Software Engineering and Artificial Intelligence.
Accepté pour communication à VERIFAI 2026 (https://www.laser-foundation.org/verifai-26), 8-11 mars 2026.

Plan

1 Introduction
2 IA et preuve formelle
3 Accompagnement de la Recherche par Intelligence Artificielle
4 Reasoned use of Artificial Intelligence by Software Engineers and Researchers
5 Découverte de configurations quantiques contextuelles optimales
6 Problèmes SAT symétriques
7 Bijections réciproques
8 Chemins auto-évitants
9 Outils du chercheur

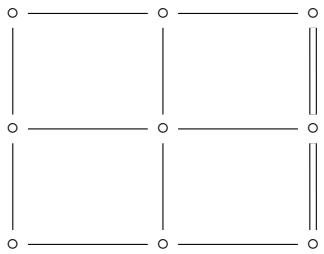
Jeu de logique (1/3)



Comment colorier certains points en noir afin que chaque ligne simple en contienne un nombre pair et chaque ligne double en contienne un nombre impair ?

Jeu de logique (2/3)

(Même question avec la ligne horizontale inférieure en plus.)



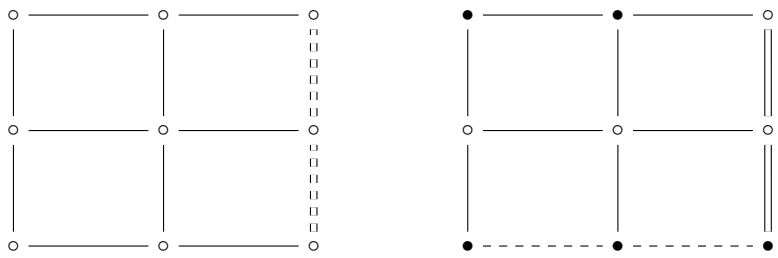
- *Comment colorier certains points en noir afin que chaque ligne simple en contienne un nombre pair et chaque ligne double en contienne un nombre impair ?*
- C'est impossible.
- *Pourquoi ?*
- Comptons le nombre de points noirs ligne par ligne : 5 lignes simples $\times$ un nombre pair $+$ 1 ligne double $\times$ un nombre impair $=$ un nombre impair. Or chaque point appartient à 2 lignes, donc chaque point noir est ainsi compté 2 fois. Ce nombre devrait être pair, contradiction. $\square$

Quel est le lien entre ce jeu et le phénomène de contextualité quantique ?

Jeu de logique (3/3)

Relaxation : quand il n'y a pas de solution, quelles sont les "meilleures" quasi-solutions ?

Comment colorier certains points en noir afin de maximiser le nombre total de lignes satisfaites ? (lignes simples avec un nombre pair et lignes doubles avec un nombre impair)

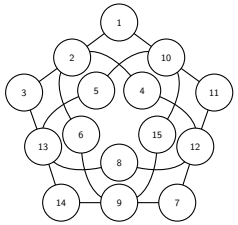


Le nombre minimal de lignes non satisfaites est appelé *degré (de contextualité)*

Comment trouver d'autres énigmes de ce genre ?

Qontextium

Qontextium (`https://quantcert.github.io/contextualityDegree`) computes the contextuality degree of *hypergrams* and *quantum configurations*



Manual encoding →

```
Hypergraph
  1,2,3
  14,9,7
  7,12,11
  11,10,1
  3,13,14
  8,13,5
  4,12,8
  15,9,6
  5,10,15
  6,2,4
Anticommutation
  matrix
011010010101101
100101101010010
010010010101101
101101010010010
010101101010010
101010010101101
010101101010010
101010010101101
010101101010010
101010010101101
```

Qontextium : Labeling + contextuality degree →

```
Contextuality
degree: 1
```

(crédit : Axel Muller, Journées du GT IQ 2025, 29 janvier 2025)

Qatlas

Découverte de configurations quantiques contextuelles optimales
Comment découvrir les meilleures structures de degré de contextualité 2, 3, 4, etc ?

Comment maximiser le degré en minimisant les nombres de points et/ou de lignes ?

1. Exécuter Qontextium sur des exemples
2. Programmer (en C ?) la génération automatique de nombreuses structures
3. Tenir compte des symétries
4. Utiliser l'IA générative (raisonnablement)
5. Présenter les résultats sous la forme d'un atlas/catalogue des structures trouvées

Concevoir un dispositif de collecte des meilleures solutions ?

Contact : `wilfried_junior.kamga_djidjou@edu.univ-fcomte.fr`

Plan

1 Introduction
2 IA et preuve formelle
3 Accompagnement de la Recherche par Intelligence Artificielle
4 Reasoned use of Artificial Intelligence by Software Engineers and Researchers
5 Découverte de configurations quantiques contextuelles optimales
6 Problèmes SAT symétriques
7 Bijections réciproques
8 Chemins auto-évitants
9 Outils du chercheur

Problèmes SAT symétriques

1. Quelle est la structure générale d'une formule en CNF sur n variables, totalement symétrique, c'est-à-dire stable par l'action de toute permutation de ses n variables ?
2. Pour chacune de ses structures, comment caractériser les valuations qui la satisfont ?
3. En déduire la forme générale des valuations-solutions
4. Combien y a-t-il de telles formules et de telles valuations-solutions ? De telles formules ?
5. Comment démontrer formellement ces résultats ?
6. Comment l'IA symbolique ou générative peut aider à trouver ou vérifier ces réponses ?

Plan

1 Introduction
2 IA et preuve formelle
3 Accompagnement de la Recherche par Intelligence Artificielle
4 Reasoned use of Artificial Intelligence by Software Engineers and Researchers
5 Découverte de configurations quantiques contextuelles optimales
6 Problèmes SAT symétriques
7 Bijections réciproques
8 Chemins auto-évitants
9 Outils du chercheur

Bijections réciproques

- Théories de couples de fonctions mutuellement inverses [Pau24]
- Autre représentation d'une permutation (et de son inverse)

[Pau24] Nicolas Pauchet.
Preuve formelle de tranformations bijectives réciproques entre permutations et factorielles avec Why3.
Master's thesis, Université de Franche-Comté, 2024.

Factorial arrays and permutations

A word $f_{n-1} \dots f_0$ of length n is a *factorial (number)* if $0 \leq f_i \leq i$ for $0 \leq i \leq n-1$ [Lai88].

23110 and 741532010 are factorials, 1 and 3020 aren't

Factorials encode permutations $\rightsquigarrow$ *let us prove it formally !*
A *permutation code* is a (size-preserving) bijection from permutations to factorials

1. Many of them are known $\rightsquigarrow$ bibliographic study
2. Formal soundness proof (with Why3) for one (de ?)coding function [GDL19]

[Lai88] C.-A. Laisant.
Sur la numération factorielle, application aux permutations.
In *Bulletin de la S. M. F., tome 16*, pages 176–183, 1888.

[GDL19] Alain Giorgetti, Catherine Dubois, and Rémi Lazarini.
Combinatoire formelle avec Why3 et Coq.
In *Journées Francophones des Langages Applicatifs (JFLA 2019)*, pages 139–154, 2019.https://hal.inria.fr/hal-01985195.

Lehmer code, formally

- Fonctions de codage et de décodage de permutations de D. H. Lehmer [Leh60]
- Preuves formelles de réciprocité, avec Why3 [Pau24]
- Complexes car programmes impératifs (nombreux invariants)

⇝ 💡 Preuve de programmes récursifs équivalents ?
⇝ ⇝ 💡 Certified lexicographic generator of factorials ?
⇝ ⇝ 💡 Application to (uniform or random) generation ?

[Leh60] D.H. Lehmer.
Teaching combinatorial tricks to a computer.
In *Proceedings of Symposia in Applied Mathematics. Combinatorial Analysis*, volume 10. American Mathematical Society, 1960.
https://api.semanticscholar.org/CorpusID:115452165.

[Pau24] Nicolas Pauchet.
Preuve formelle de tranformations bijectives réciproques entre permutations et factorielles avec why3.
Master's thesis, Université de Franche Comté, 2024.
Defended the 17 June 2024.

Ingrédients

- Découverte de Why3 en autonomie (vidéos + site web + exemples + articles)
- Travaux de 2 équipes M1 de 2024-25 fournis
- Preuve formelle à finir

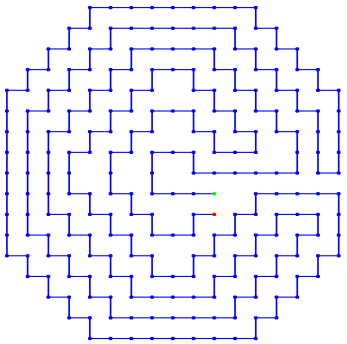
S'il reste du temps, application à d'autres couples de fonctions de codage et de décodage

Plan

1 Introduction
2 IA et preuve formelle
3 Accompagnement de la Recherche par Intelligence Artificielle
4 Reasoned use of Artificial Intelligence by Software Engineers and Researchers
5 Découverte de configurations quantiques contextuelles optimales
6 Problèmes SAT symétriques
7 Bijections réciproques
8 Chemins auto-évitants
9 Outils du chercheur

Contexte (1/2)

Un *chemin auto-évitant* (*SAW* en anglais) de longueur n est une suite de n points du plan deux à deux distincts, à coordonnées entières, telle que la distance euclidienne entre deux points successifs est 1.



w_0 (239 steps)

Contexte (2/2)

Ces chemins sont étudiés en biologie, comme des modèles simplifiés de protéines repliées.

Un chemin auto-évitant est dit *(dé)pliable* s'il peut être transformé en un autre chemin auto-évitant en faisant pivoter à angle droit autour d'un de ses points le sous-chemin qui va de ce point à l'une de ses extrémités.

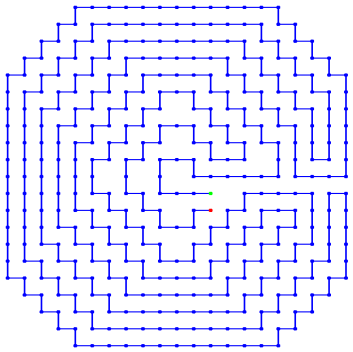
Existe-t-il des chemins auto-évitants non pliables ?

En 2013, C. Guyeux a proposé une famille infinie de chemins auto-évitants non (dé)pliables particuliers, de longueur croissante, dont le premier élément est w_0.

La démonstration de cette propriété est très longue. . .

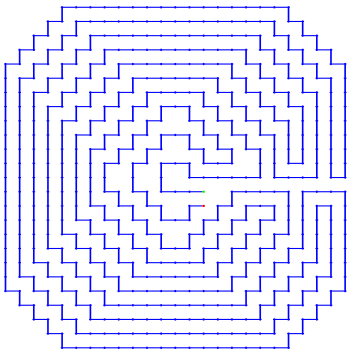
Comment faire une preuve formelle de cette propriété ? Avec quels outils d'aide à la preuve ?

Chemins suivants



w_1 (391 steps)

Chemins suivants



w_2 (575 steps)

1. Récurrence connue, fournie
2. Codage Prolog fourni (résultat M1 IR 2024-25)
3. Formaliser en Why3 ? En KeY ?

Plan

1 Introduction
2 IA et preuve formelle
3 Accompagnement de la Recherche par Intelligence Artificielle
4 Reasoned use of Artificial Intelligence by Software Engineers and Researchers
5 Découverte de configurations quantiques contextuelles optimales
6 Problèmes SAT symétriques
7 Bijections réciproques
8 Chemins auto-évitants
9 Outils du chercheur

Quelques outils du chercheur

Génériques

- Travail collaboratif : outils d'édition scientifique collaborative
- Veille scientifique : Zotero
- Rédaction, visualisation, communication : LaTeX, beamer, listings, tikz, . . .

Spécifiques (IA symbolique)

- Clauses de Horn : Prolog
- Formule propositionnelle : solveurs SAT
- Raisonnement modulo théories : solveurs SMT
- Plateforme de vérification déductive : Why3

Édition scientifique collaborative

`https://plmlatex.math.cnrs.fr/`

- Rédaction collaborative en LaTeX, en ligne
- Connexion avec compte ENT
- Alternatives (pas testées, plus risqué) : `https://typst.app/`, `https://cocalc.com`

Usages dans ce module

1. Création d'un projet par GT
2. Partage avec alain.giorgetti@univ-fcomte.fr, édition autorisée
3. Rédaction (avec LaTeX) d'une note de lecture, citant les références bibliographiques liées au sujet de recherche choisi
4. Préparation de slides de présentation des résultats du groupe (avec la classe beamer de LaTeX)
5. Gestion de projet allégée

⇝ Récupérer sur Moodle des modèles de ces deux fichiers

Démonstration . . .

Gestion de projet allégée

Appliquer les principes des méthodes agiles à ce mini-projet de recherche

- 1 sprint par semaine
- Coordination lors des TDs et TPs
- Avec quels outils ? Solution légère : planification/affectation/suivi des tâches par sections et sous-sections LaTeX dans une annexe du rapport de GT

Démonstration . . .

⇝ Inclure l'historique du projet dans le rapport de synthèse final, en annexe

Gestion de références bibliographiques avec Zotero

`http://www.zotero.org`

- Collecte, tri et partage de références bibliographiques
- Libre, facile à installer et à utiliser (TP)
- Objectifs
  1. Identifier des références bibliographiques liées au sujet de recherche choisi
  2. Collecter ces références dans une bibliothèque de groupe (de préférence avec un PDF)
  3. Extraire un fichier `.bib`
  4. Survoler ces références pour y voir les liens avec le sujet
  5. Rédiger (en LaTeX) une description de ces liens en citant ces références bibliographiques (`\cite{..}`)

Démonstration . . .

Recherche bibliographique

Accès au texte intégral (PDF)

- Archives ouvertes
  - `https://arxiv.org` (+ sources LaTeX)
  - `https://hal.archives-ouvertes.fr`, `http://www.theses.fr/`
  - `https://www.researchgate.net`
- Click & Read CNRS (`https://clickandread.inist.fr`)

Outils d'assistance

- `https://scholar.google.com`
- `https://dblp.org/`
- `https://www.connectedpapers.com`
- ...

Autres sources d'information

- `https://fr.wikipedia.org`

Formalization and deductive verification with Why3

`https://www.why3.org`

- Programming and specification language WhyML, with functional and imperative features
- Polymorphic and algebraic types
- Standard library : int, lists, arrays, ...
- Verification condition generator for SMT solvers (Alt-Ergo, CVC3, CVC4, Z3) and proof assistants (e.g. Coq)
- Extraction to OCaml code

$\rightsquigarrow$ Avant le cours 2, découverte en autonomie avec la présentation vidéo de S. Blazy : `https://panopto.abo.fi/Panopto/Pages/Viewer.aspx?id=63fe9f17-4ee7-4bd1-b9e4-adad00fd64ac`

Ordre lexicographique strict sur les listes d'entiers, en WhyML

Exemple : listes de taille 3 bornées par 2

$[0,0,0] \prec [0,0,1] \prec [0,1,0] \prec [0,1,1] \prec [1,0,0] \prec [1,0,1] \prec [1,1,0] \prec [1,1,1]$

Implémentation

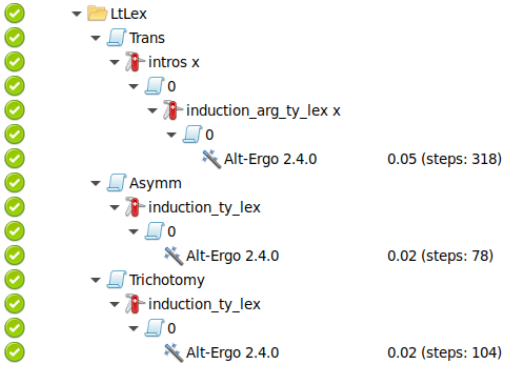
```
let rec predicate lt_lex (l1 l2: list int) =
  match l1 with
  | Nil →
    match l2 with | Nil → false | _ → true end
  | Cons x1 r1 →
    match l2 with
    | Nil → false
    | Cons x2 r2 → x1 < x2 || x1 = x2 && lt_lex r1 r2
    end
  end

clone relations.TotalStrictOrder with type t = list int,
  predicate rel = lt_lex
```


Exemple de preuve d'ordre total strict, avec Why3

Preuve que la fonction `lt_lex` respecte les propriétés d'un ordre total strict

Transitivité :	$\forall\, l_1, l_2, l_3.$	$l_1 \prec l_2 \wedge l_2 \prec l_3$	$\rightarrow$	$l_1 \prec l_3$
Asymétrie :	$\forall\, l_1, l_2.$	$l_1 \prec l_2$	$\rightarrow$	$\neg(l_2 \prec l_1)$
Trichotomie :	$\forall\, l_1, l_2.$	$l_1 \prec l_2 \vee l_2 \prec l_1 \vee l_1 = l_2$		



Prolog pour énumérer et compter

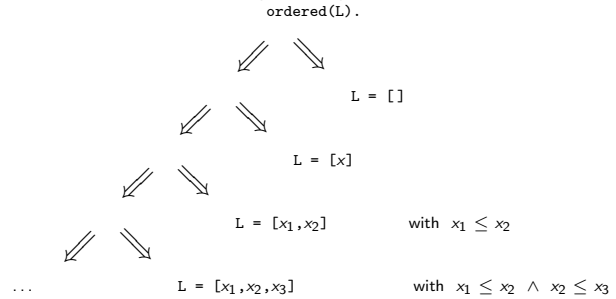
Programmation logique / logic programming (LP)

- Programs are sets of rules (Horn clauses) of the form
  $\mathtt{H}$:- $\mathtt{A}_1 \wedge \ldots \wedge \mathtt{A}_n$
  (meaning, $\mathtt{H}$ holds if $\mathtt{A}_i$ holds for $i = 1, \ldots, n$)
- Example
  ordered([]).
  ordered([x]).
  ordered([x_1,x_2|L]) :- $x_1 \leq x_2 \wedge$ ordered([x_2|L]).
- Query evaluation
  1. Pick leftmost atom in current query : $\mathtt{Q} = \mathtt{A} \wedge \mathtt{R}$
  2. Find unifying head : $\mathtt{A}\sigma = \mathtt{H}\sigma$
  3. Rewrite to get a new query : $(\mathtt{A}_1 \wedge \ldots \wedge \mathtt{A}_n \wedge \mathtt{R})\sigma$

LP-based exhaustive generation

ordered([]).
ordered([x]).
ordered([x_1,x_2|L]) :- $x_1 \leq x_2 \wedge$ ordered([x_2|L]).

can work as a generator (for free) :



Advantages of Logic Programming

- Specification is executable
- Many available techniques, such as *program transformations* → optimization by filter promotion (e.g. [Senni and Fioravanti, TAP'12], constraint-based)
- For instance, validation of an efficient algorithm wrt a simpler specification (naive implementation)
- General 'differential testing' framework :

generator	*acceptor*
specification	implementation
implementation	specification
implementation$_1$	implementation$_2$
⋮	⋮

Property checking by counting

```
:- compile(measure).
```

- swipl library written by Valerio Senni [GS12], distributed in CUT [DG18]
- Example (binary trees)

```
bin(leaf,0).
bin(node(T1,T2),N) :-
  Nm1 is N-1, in(N1,0,Nm1), N2 is Nm1-N1,
  bin(T1,N1), bin(T2,N2).
```

- First numbers

```
?- iterate(0,10,bin).
```

⇝ 1,1,2,5,14,42,132,429,1430,4862,16796 (in less than 1 second)
https://oeis.org/A000108

[GS12] Alain Giorgetti and Valerio Senni.
Specification and validation of algorithms generating planar Lehman words.
In *GASCom'12, 8-th Int. Conf. on random generation of combinatorial structures*, Bordeaux, France, June 2012.
https://hal.inria.fr/hal-00753008.

[DG18] Catherine Dubois and Alain Giorgetti. Tests and proofs for custom data generators.
Formal Aspects of Computing, 30 :659–684, Jul 2018. https://doi.org/10.1007/s00165-018-0459-1.